

Quiz

1

Let $G = (V, E)$ be *connected* and assume all vertices of G have even degree. Recall the algorithm `Euler_Walk` from the lecture:

`Euler_Walk(u)`:

if there exists an edge $\{u, v\}$ which is not marked:

mark the edge $\{u, v\}$

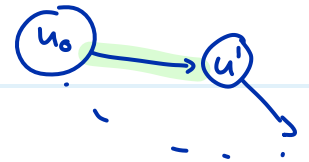
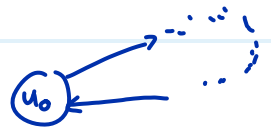
`Euler_Walk(v)`

topological sorting?

12 min

Suppose we start the recursion with a call to `Euler_Walk( $u_0$ )` for some vertex $u_0 \in V$.

True	False	
✗		We have an even number of marked edges which contain u_0 immediately after the start of any call to <code>Euler_Walk(u_0)</code> .
	✗	Let $u' \in V$ with $u' \neq u_0$, then we have an even number of marked edges which contain u' immediately after the start of any call to <code>Euler_Walk(u')</code> .



Suppose that $G = (V, E)$ has an Eulerian Walk. Then we can always find an Eulerian Walk of G in time $O(|V|)$.

True

☒ False

Algorithmus in $O(|E|)$

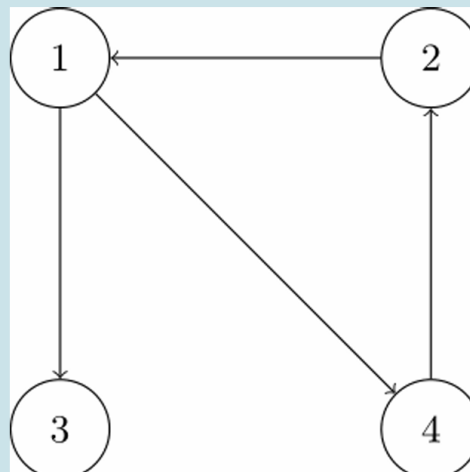
falls $|E| = \frac{n \cdot (n-1)}{2}$, dann $O(|E|) \neq O(|V|)$

What adjacency matrix describes the graph above? The indices are in increasing numerical order.

a. $\begin{pmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$

b. $\begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$

c. $\begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$



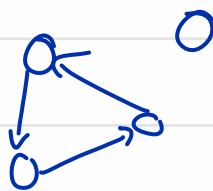
Suppose we restrict our inputs to graphs $G = (V, E)$ which are *connected* and satisfy $|E| \geq |V|^2/10$, rather than the class of all graphs. Then, depth-first search (DFS) runs (in the worst case over such graphs) asymptotically in the same time regardless of whether the graph is stored as adjacency lists or as an adjacency matrix.

$$\text{True} \leq O\left(\frac{|V|^2}{10} + |V|\right)$$

$$O(|V|^2)$$

$$\text{False} \leq O(|V|^2)$$

Let G be a directed graph that contains a (directed) cycle. Which of the following statements are true for all such G ?

True	False			
	✗	G has no source vertex.		
	✗	G has no sink vertex.		
✗		G does not have a topological sorting of its vertices.		

Which vertex is at the end of every topological sorting of the graph above?

1. 1

2. 2

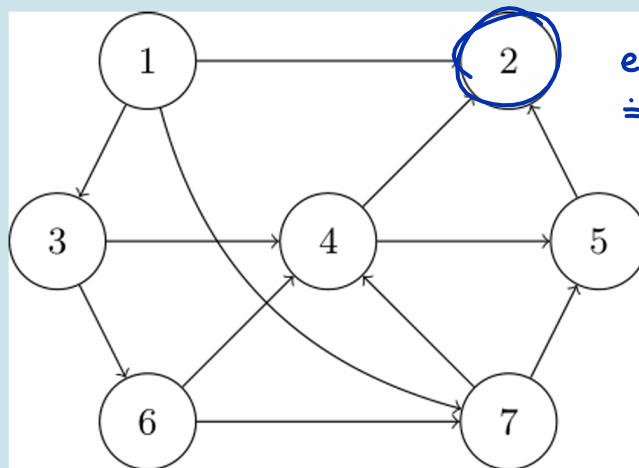
3. 3

4. 4

5. 5

6. 6

7. 7



einzigste Senke
 $\Rightarrow$ muss am Schluss sein

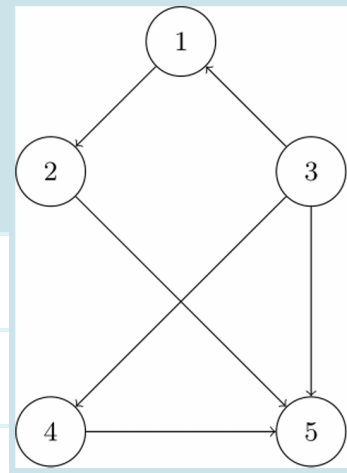
1, 3, 6, 7, 4, 5, 2

In every topological sorting of the graph above, vertex 1 must come before vertex 4.

True

False

3, 4, 1, 2, 5



We run *depth-first-search* (DFS) on a directed graph $G = (V, E)$ which contains a directed cycle. We determine all the *pre/post numbers*.

What can we conclude about the relationship of pre/post numbers?

True	False			
	X	For every edge $(u, v) \in E$, we have $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$. all edges must be back edges		
X		There exists some edge $(u, v) \in E$ which has $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$. there must be at least one back edge because G contains a directed cycle		

We run *depth-first-search* (DFS) on a directed graph $G = (V, E)$ and we determine all the *pre/post numbers*. Then for an edge $(u, v) \in E$, it's possible that $\text{pre}(u) < \text{pre}(v) < \text{post}(u) < \text{post}(v)$.

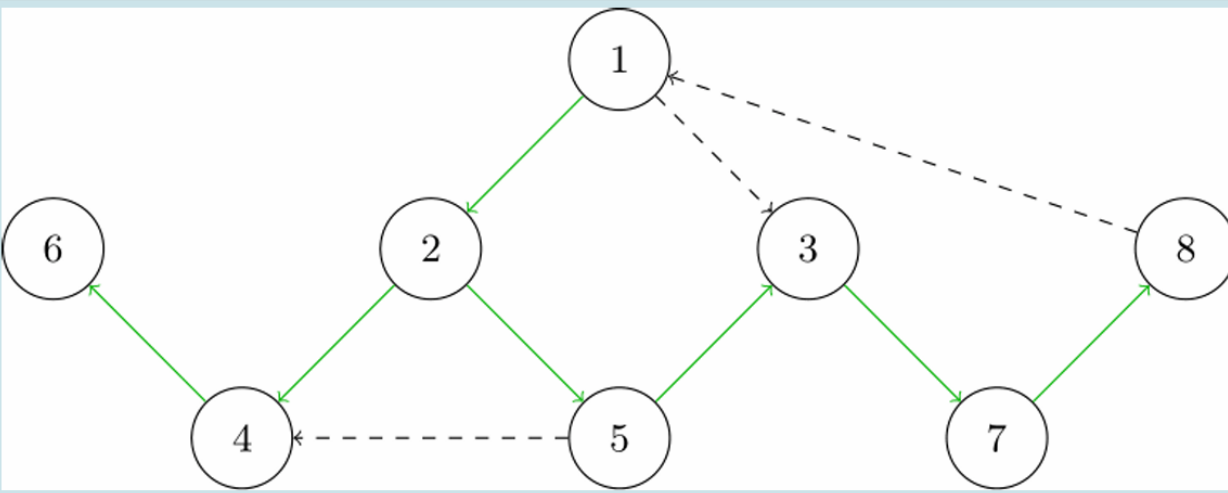
True

False

v is child of u

parent u only finishes if all children are visited





In the graph above, the green, solid edges indicate a *depth-first search tree*. Classify the remaining edges. (Hint: each option should be used **exactly once**).

(5, 4)	Cross Edge	▼
(8, 1)	Back Edge	▼
(1, 3)	Forward Edge	▼

DP Notation (Beispiel Subset Sum)

Dimension: $T[0 \dots n][0 \dots s]$ oder $T[0 \dots n, 0 \dots s]$ oder $(n+1) \times (s+1)$

$$\begin{aligned}
 \text{Teilproblem: } T[i, s] = T[i][s] &= \begin{cases} 1 & \text{falls } A \\ 0 & \text{sonst} \end{cases} \\
 &= \begin{cases} \text{true} & \text{falls } A \\ \text{false} & \text{sonst} \end{cases} \\
 &= \begin{cases} 1 & \text{falls } A \\ 0 & \text{falls } \neg A \end{cases}
 \end{aligned}$$

Base case: $T[0,0] = 1$ oder $T[i,0] = 1 \quad \forall 0 \leq i \leq n$

$$T[0, s] = 0 \quad \forall 1 \leq s \leq b$$

Rekursion: $T[i, s] = \underbrace{T[i-1, s]}_{\text{Wir nehmen } A[i] \text{ nicht}} \vee \underbrace{T[i-1, s-A[i]]}_{\text{Wir nehmen } A[i]} \quad \text{falls } A[i] > s$

$\forall 1 \leq i \leq n \quad \forall 0 \leq s \leq b$

oder $\forall 1 \leq i \leq n \quad \forall 1 \leq s \leq b$
 $\uparrow$
 $s=0$ ist ein base case

bitte nicht in Java: $T[i][s] = T[i-1][s] \parallel (T[i-1][s-A[i]] \ \&\& \ s \geq A[i]) \rightarrow \text{Index Fehler}$

$$T[i][s] = T[i-1][s] \parallel (s \geq A[i] \ \&\& \ T[i-1][s-A[i]])$$

Justify your recursion!

Berechnungsreihenfolge:

<pre> For i = 0 ... n: For s = 0 ... b: compute T[i,s] </pre> oder 1	<pre> For i = 0 ... n: For s = 0 ... b: compute T[i,s] </pre> oder 1
---	---

was genau hier steht ist nicht so wichtig, die Loops sind entscheidend

→ entweder beide bei 1 oder beide bei 0 anfangen

Laufzeit: $O(n \cdot b)$

Mustersatz: Es gibt $(n+1)(b+1)$ DP Einträge, wobei jeder einzelne Eintrag in $O(n)$ berechnet werden kann, was eine Gesamtlaufzeit von $O(n^2 b)$ ergibt.

Die Lösung kann in $O(1)$ ausgelesen werden

Gerichtete Graphen

$$G = (V, E), \quad E \subseteq \{(u, v) \mid u, v \in V, u \neq v\}$$

$\deg_{\text{in}}(v)$ = Anzahl eingehender Kanten

$\deg_{\text{out}}(v)$ = Anzahl ausgehender Kanten

Quelle falls $\deg_{\text{in}}(v) = 0$



Senke falls $\deg_{\text{out}}(v) = 0$



Weg, Zyklus, Pfad, Kreis analog wie bei ungerichteten Graphen, mit der zusätzlichen Bedingung, dass man nur in Richtung der Kante gehen darf

Wie viele gerichtete Pfade von e nach c? 2

Wie viele gerichtete Pfade von c nach e? 1

Wie viele gerichtete Wege von c nach e? ∞

Wie viele gerichtete Kreise? 2

Was sind alle möglichen Längen gerichteter Wege von d

nach e? $\{3 + 4n \mid n \in \mathbb{N}\}$

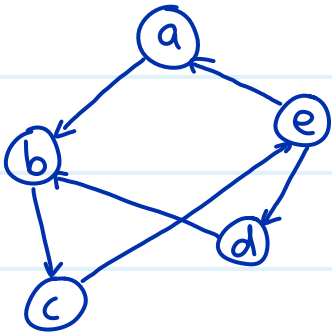
gerichteter Hamiltonpfad? ✓

gerichteter Hamiltonkreis? ✗

zusammenhängend? ✓

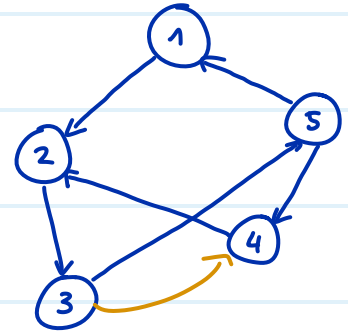
gibt es einen gerichteten Kreis der a und d enthält? ✗

gibt es einen gerichteten Zyklus der a und d enthält? ✓



Adjazenzmatrix: $A_{u,v} = 1 \Leftrightarrow (u,v) \in E$

	1	2	3	4	5
1	0	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1
4	0	1	0	0	0
5	1	0	0	1	0



Kante einfügen/löschen in $O(1)$

Alle Nachfolger von v aufzählen in $O(n)$

Teste ob $(u,v) \in E$ in $O(1)$

Adjazenzliste: $A[u]$ speichert Liste aller Nachfolger von u in beliebiger Reihenfolge

1	[2],
2	[3],
3	[5, 4],
4	[2],
5	[1, 4]

Kante (u,v) einfügen/löschen in $O(\text{degout}(u) + 1)$ (wir prüfen zuerst ob die Kante schon existiert)

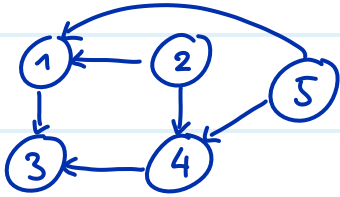
Alle Nachfolger von u aufzählen in $O(\text{degout}(u) + 1)$

Teste ob $(u,v) \in E$ in $O(\text{degout}(u) + 1)$

Handschlag Lemma für gerichtete Graphen: $\sum_{v \in V} \text{deg}_{\text{in}}(v) = \sum_{v \in V} \text{deg}_{\text{out}}(v) = |E|$

Topologische Sortierung

$v_1, v_2, v_3, \dots, v_n$ wobei alle Vorgänger von v_i vorher in der Sortierung stehen



finde alle möglichen topologischen Sortierungen

2, 5, 1, 4, 3

2, 5, 4, 1, 3

5, 2, 1, 4, 3

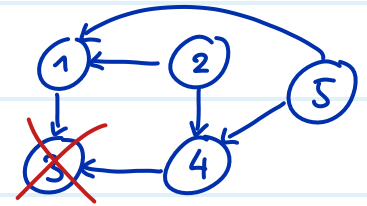
5, 2, 4, 1, 3

Algorithmus:

finde Senke s

setze s an die letzte Stelle der top Sortierung

lösche s und inzidente Kanten und löse den Rest rekursiv



$\exists$ topologische Sortierung $\Leftrightarrow \nexists$ gerichteter Zyklus

$$A \rightarrow B \equiv \neg B \rightarrow \neg A$$

Beweis " $\Rightarrow$ " indirekt: $\exists$ gerichteter Zyklus $\Rightarrow \nexists$ top Sortierung

Sei $(v_1, v_2, \dots, v_k, v_1)$ ein Zyklus. Dann muss v_1 vor v_k stehen und

v_k vor v_1 . Das geht nicht. Deshalb gibt es keine top Sortierung

" $\Leftarrow$ " direkt: $\nexists$ Senke $\Rightarrow \exists$ gerichteter Zyklus

(jeder Knoten hat einen Nachfolger. Wir gehen solange zum Nachfolger bis wir einen bereits besuchten Knoten finden, und somit einen gerichteten Zyklus gefunden haben)

$\Rightarrow \nexists$ gerichteter Zyklus $\Rightarrow \exists$ Senke

$$\leftarrow A \rightarrow B \equiv \neg B \rightarrow \neg A$$

$\Rightarrow$ Algorithmus funktioniert

$\Rightarrow \exists$ top Sortierung

DFS

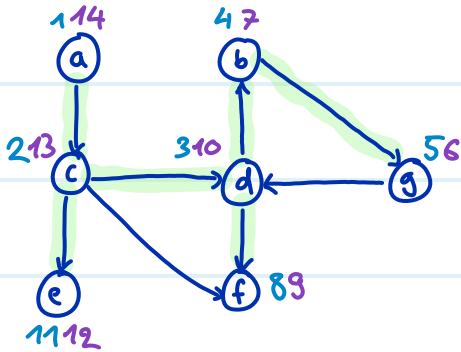
DFS(G) (Rahmenprogramm wird gebraucht falls G nicht zusammenhängend ist)

- 1 for each $v \in V$ do
- 2 if v ist noch nicht besucht then
- 3 DFS-VISIT(G, v)

DFS-VISIT(G, v)

- 1 Markiere v als besucht
- 2 for each $(v, w) \in E$ do
- 3 if w ist noch nicht besucht then
- 4 DFS-VISIT(G, w)

DFS mit Startknoten a

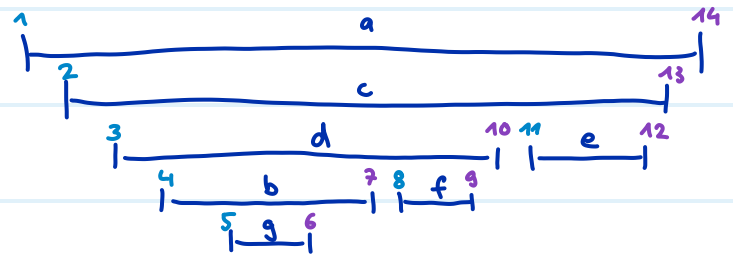
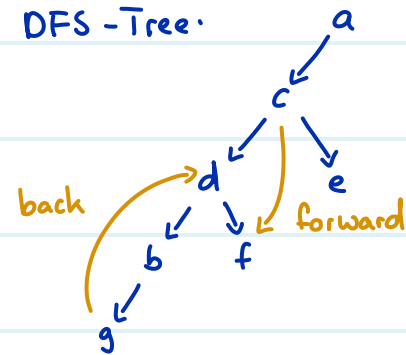


Post (Knoten fertig)

Pre (Knoten Anfang)

Tree edges

DFS-Tree.



Kante (u, v)

Forward edge: $\begin{array}{|c|} \hline u \\ \hline v \\ \hline \end{array}$ und nicht tree edge $\Rightarrow$ vorwärts im DFS-Tree

Back edge: $\begin{array}{|c|} \hline v \\ \hline u \\ \hline \end{array}$ $\Rightarrow$ rückwärts im DFS-Tree

Cross edge: $\begin{array}{|c|} \hline v \\ \hline \end{array} \begin{array}{|c|} \hline u \\ \hline \end{array}$ $\Rightarrow$ wechselt Teilbaum im DFS-Tree

andere Fälle sind nicht möglich!

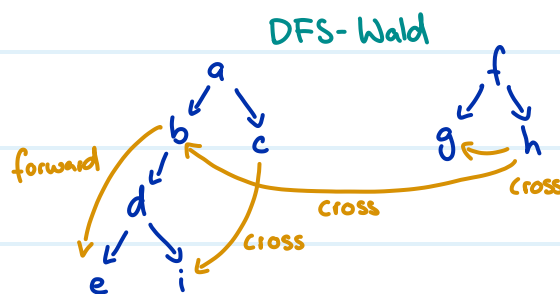
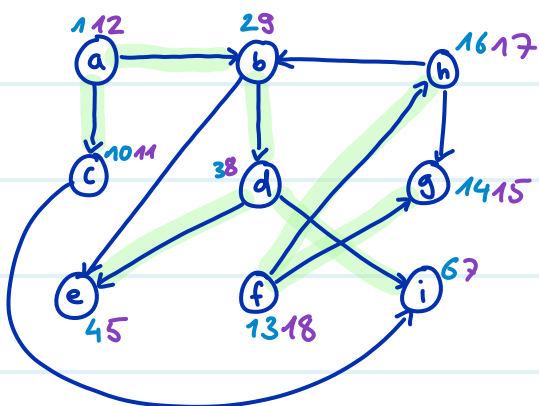
$\exists$ back edge $\Leftrightarrow \exists$ gerichteter Zyklus

$\nexists$ back edge $\Rightarrow$ umgekehrte Post-order ist eine topologische Sortierung

Laufzeit: Adjazenzmatrix: $O(n^2)$

Adjazenzliste: $O(n+m)$ $\Rightarrow$ nimmt immer Adjazenzlisten!

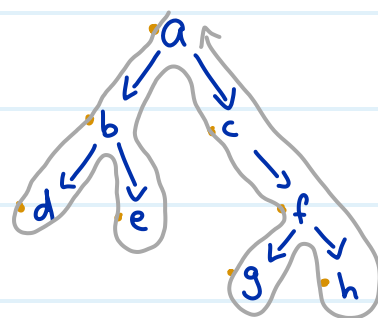
DFS mit Startknoten a, bei mehreren Möglichkeiten alphabetisch



Trick für pre/post-order:

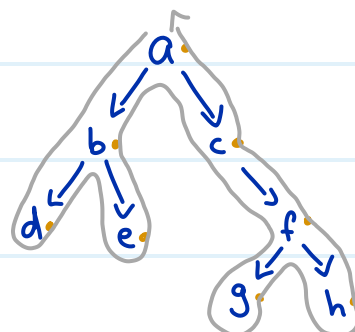
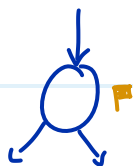
DFS-Baum ablaufen

Flagge links
= Pre



a, b, d, e, c, f, g, h

Flagge rechts
= Post

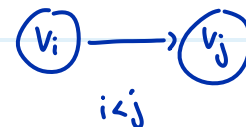


d, e, b, g, h, f, c, a

Exercise 9.4 Longest path in a DAG (1 point). Bonus HS24

Let $G = (V, E)$ be a directed graph without directed cycles (i.e., a directed acyclic graph or short DAG). Assume that $V = \{v_1, \dots, v_n\}$ (for $n = |V| \in \mathbb{N}$) and that the sorting $v_1, v_2, \dots, v_n$ of the vertices is a topological sorting. The goal of this exercise is to find the longest path in G .

(a) Let P be a path in G . Prove that if $P = (v_{i_1}, v_{i_2}, \dots, v_{i_k})$, then $i_1 < i_2 < \dots < i_k$.



P besteht aus den Kanten $(v_{i_1}, v_{i_2}), (v_{i_2}, v_{i_3}), \dots, (v_{i_{k-1}}, v_{i_k})$

$\Rightarrow i_1 < i_2, i_2 < i_3, \dots, i_{k-1} < i_k$ (top. Sortierung)

$\Rightarrow i_1 < i_2 < i_3 < \dots < i_{k-1} < i_k$

(b) Describe a bottom-up dynamic programming algorithm that, given a graph G with the property that $v_1, \dots, v_n$ is a topological sorting, returns the length of the longest path in G in $O(|V| + |E|)$ time. You can assume that the graph is provided to you as a pair (n, Adj) of the integer $n = |V|$ and the adjacency lists Adj . Your algorithm can access $Adj[u]$, which is a list of vertices to which u has a direct edge, in constant time. Formally, $Adj[u] := \{v \in V \mid (u, v) \in E\}$.

In your solution, address the following aspects:

1. *Dimensions of the DP table:* What are the dimensions of the DP table?
2. *Subproblems:* What is the meaning of each entry?
3. *Recursion:* How can an entry of the table be computed from previous entries? Justify why your recurrence relation is correct. Specify the base cases of the recursion, i.e., the cases that do not depend on others.
4. *Calculation order:* In which order can entries be computed so that values needed for each entry have been determined in previous steps? Describe the calculation order in pseudocode.
5. *Extracting the solution:* How can the solution be extracted once the table has been filled?
6. *Running time:* What is the running time of your solution?

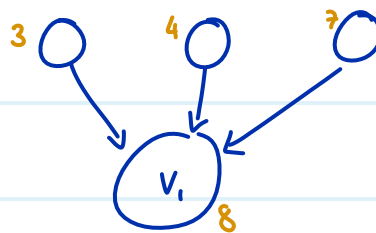
Hint: Define the entry of the DP table as $DP[i] = \text{length of longest path in } G \text{ ending at vertex } v_i$.

Dimension: $DP[1 \dots n]$

Teilproblem: $DP[i] = \text{length of a longest path in } G \text{ ending at vertex } v_i$

Base case: $DP[1] = 0$ weil v_1 eine Quelle ist, d.h. $\deg_{\text{in}}(v_1) = 0$, d.h. kein Pfad kann in v_1 enden

Rekursion:
$$DP[i] = \begin{cases} 0 & \text{falls } v_i \text{ eine Quelle ist} \\ 1 + \max_{1 \leq j < i} \{DP[j] \mid (v_j, v_i) \in E\} & \text{sonst (} v_i \text{ hat Vorgänger)} \end{cases}$$
 $\forall 2 \leq i \leq n$



1 + max von Vorgängern

Berechnungsreihenfolge: For $i = 1 \dots n$:

compute $DP[i]$

d.h. topologische Sortierung aufsteigend

Lösung: $\max_{1 \leq i \leq n} \{DP[i]\}$ und dann backtracking

Laufzeit: Trick: Adjazenzliste umdrehen (möglich in $O(n+m)$)

Berechnung für $DP[i] \leq O(1 + \deg_{in}(v_i))$

$$\text{Gesamter DP-Table: } \sum_{i=1}^n O(1 + \deg_{in}(v_i)) = O(n) + \sum_{i=1}^n O(\deg_{in}(v_i))$$

$$= O(n+m) \quad \text{Handschlaglemma}$$

Lösung auslesen in $O(n)$

back tracking in $O(n)$ wir merken uns den Vorgänger während dem Berechnen der DP-Tabelle

Write a Java program that counts the number of **connected components** in an undirected graph represented as an adjacency list.

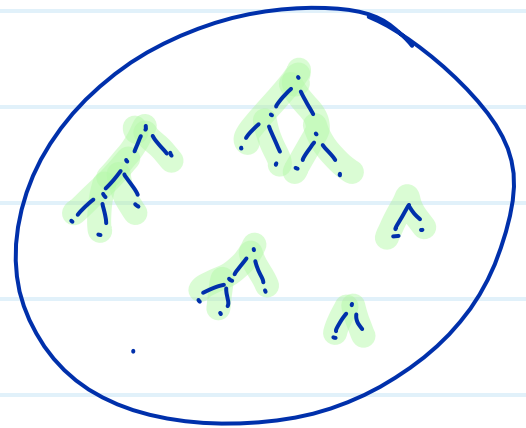
```
public static int countConnectedComponents(int[][] G) {
    int n = G.length;
    boolean[] visited = new boolean[n];
    int componentCount = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            componentCount++;
            dfs(i, G, visited); // dfs and bfs both work
        }
    }

    return componentCount;
}
```

pass by reference

= Anzahl Bäume im DFS Wald



```
public static void dfs(int x, int[][] G, boolean[] visited) {
    visited[x] = true; // Mark the current node as visited

    for (int i = 0; i < G[x].length; i++) { // iterate through all neighbours of x
        int y = G[x][i]; // get the i-th neighbour
        if (!visited[y]) { // if we haven't visited y
            dfs(y, G, visited); // then we visit y
        }
    }
}
```